



Vector Databases e Big Data

- Lezione 1 - Prof. Giuseppe Fenza

Cosa sono i Vector Databases (VDBs)



- Strumenti progettati per **memorizzare, gestire e ricercare** vettori ad alta dimensione (*embeddings*)
- Ottimizzati per lavorare con **dati complessi e non strutturati** (testo, immagini, audio, video)

 **Obiettivo:** *permettere ricerche basate sulla similarità semantica, non solo su valori esatti o chiavi come nei database tradizionali*

Perchè usare un VDB?

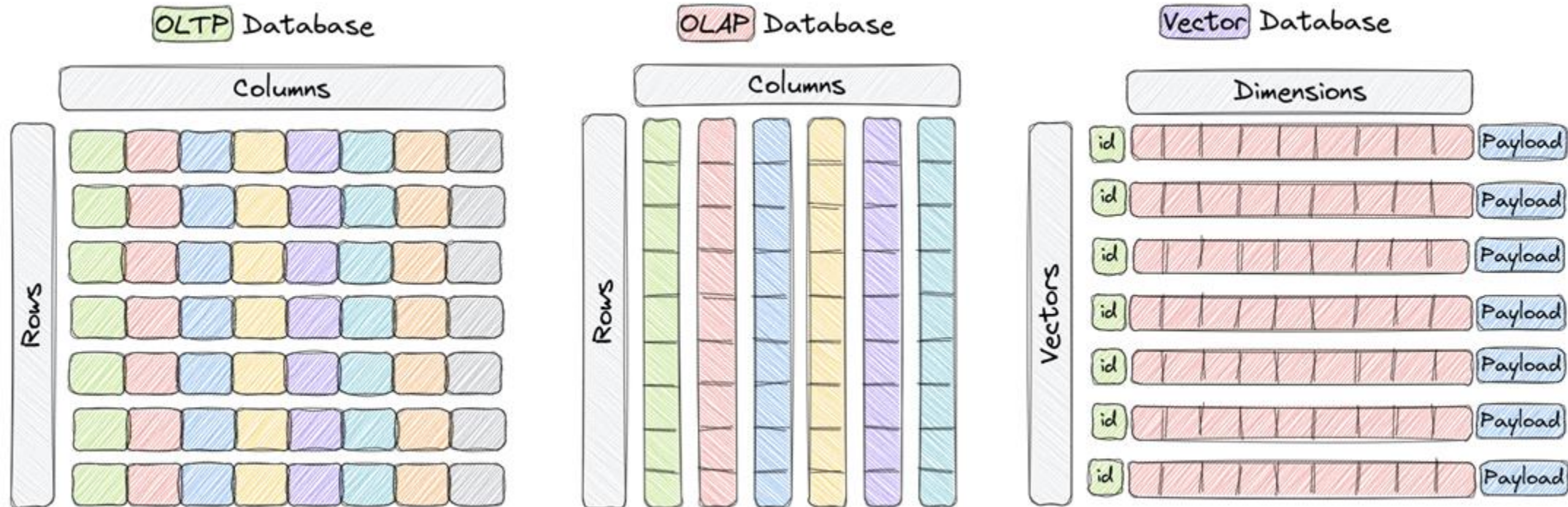


- **Retrieval di vettori efficiente e accurato**
→ capacità di trovare i vettori più simili con alta precisione
- **Supporto a dati complessi e non strutturati**
→ possibilità di memorizzare e interrogare testo, immagini, audio, video
- **Alta scalabilità e processamento real-time**
→ gestione di grandi volumi di dati con risposte immediate e possibilità di distribuire il lavoro su più macchine

Differenza tra OLAP, OLTPone e VDBs



- Nei database OLTP e OLAP tradizionali, i dati strutturati sono organizzati in Tabelle composte da record (righe) e attributi (colonne), grazie alle quali è possibile eseguire query in maniera efficiente.
- I database tradizionali gestiscono bene dati strutturati, ma sono inadeguati per la gestione e la ricerca di vettori ad alta dimensione.
- I VDBs sono, invece, progettati per archiviare e utilizzare in modo efficiente vettori ad alta dimensionalità come gli embeddings (dati non strutturati).



Pipeline



Generazione degli *embeddings*

Indicizzazione nel Vector
DB (**Storage**)

Query

Recupero dei documenti
(**Retrieval**)

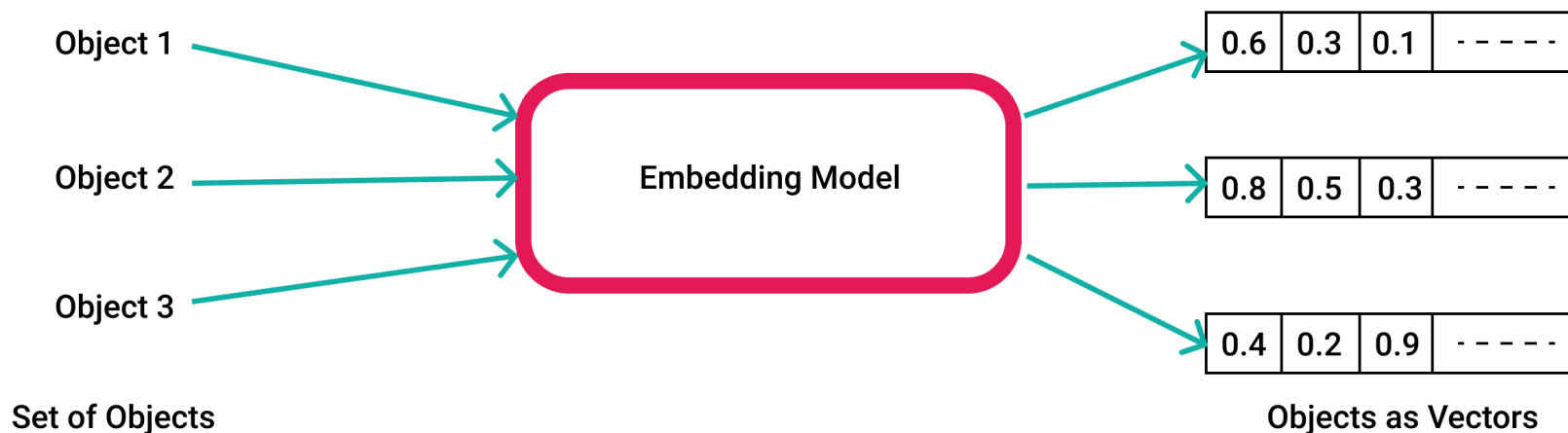
Cosa sono gli *embeddings*



All'interno di un VDB i dati grezzi vengono trasformati in *embeddings*, ovvero vettori densi con un numero fisso di dimensioni: ognuna di esse corrisponde ad una caratteristica latente o un aspetto dei dati.

- **Funzionamento**

- Un modello di machine learning prende in input un dato (testo, immagine, video, audio).
- Lo trasforma in un vettore in uno spazio ad alta dimensione (es. 768 dimensioni).
- In questo spazio, dati simili hanno vettori vicini, mentre dati diversi sono lontani.



Chunking



- È la tecnica di spezzettare un testo lungo in blocchi più piccoli chiamati *chunk*.
- Ogni chunk viene trasformato in un embedding separato.
- Permette al Vector DB di gestire testi lunghi e complessi in modo più efficace.
- Tipologie di chunking: semplice, a lunghezza fissa, e semantico.
- ***Perché si fa?***
 - **Limiti dei modelli:** molti modelli di embedding hanno un numero massimo di token (es. 512 o 1024).
 - **Precisione nel retrieval:** un unico embedding per testi lunghi può nascondere informazioni importanti.
 - **Granularità nella ricerca:** chunk più piccoli significa che il VDB può restituire parti specifiche del testo, non solo il documento intero.
 - **Contesto locale:** mantiene relazioni tra parole o frasi vicine, migliorando la similarità semantica.

Chunking semplice



- Ogni chunk rappresenta una **frase** o un **paragrafo**.
- **Vantaggio**: facile da implementare, mantiene il contesto locale.
- **Esempio**
 - Testo: “Il gatto dorme. Oggi piove. Il cane gioca.”
 - Chunk: ["Il gatto dorme.", "Oggi piove.", "Il cane gioca."]
- **Impatto**
 - Ricerca precisa su porzioni brevi di testo.
 - Se la query al VDB è specifica, il chunk corretto viene ritrovato facilmente.
 - **Limite**: si perde il contesto globale.

Chunking a lunghezza fissa



- Divisione in blocchi di **N token** o **N caratteri**, con **overlap** opzionale.
- **Vantaggio**: mantiene contesto anche per testi lunghi.
- **Esempio**
 - Chunk 1: token 0–99
 - Chunk 2: token 80–179 (overlap 20 token)
 - Chunk 3: token 160–259
- **Impatto**
 - Mantiene il contesto tra i chunk grazie all'overlap.
 - Riduce il rischio che una query non trovi il chunk corretto perché l'informazione era alla fine del chunk precedente.
 - Permette di gestire testi molto lunghi senza superare il limite di token del modello.
 - **Limite**: più chunk significa più calcolo e storage di embedding.

Chunking semantico



- Divide il testo in blocchi coerenti **semanticamente**.
- Non si basa sulla lunghezza, ma sul significato
- **Esempio:** articolo scientifico diviso in sezioni
 - Chunk 1: Introduzione
 - Chunk 2: Metodologia
 - Chunk 3: Risultati
 - Chunk 4: Discussione
 - Chunk 5: Conclusioni
- **Impatto**
 - Alta rilevanza semantica e i chunk trovati sono più pertinenti rispetto alla query.
 - Ottimo per ricerche basate sul significato, non solo sulla presenza di parole.
 - **Limite:** richiede preprocessing più complesso (analisi semantica, clustering, modelli NLP).

Comparazione tra tecniche di chunking



Tecnica	Precisione query	Contesto preservato	Complessità
<i>Semplice</i>	Media	Locale	Bassa
<i>Lunghezza fissa</i>	Alta	Medio/Alto	Media
<i>Semantico</i>	Molto alta	Alto	Alta

Le due funzionalità fondamentali dei VDBs



Vector Storage

- Gestione efficiente di grandi collezioni di vettori.
- Uso di tecniche di indicizzazione ad hoc (sharding, partitioning, caching e replication).

Vector Retrieval

- Supporto a query per trovare elementi “simili”
 - Exact Nearest Neighbors Search (NNS)
 - Approximate Nearest Neighbors Search (ANNS)

Vector Storage - Sharding



- Serve a dividere il dataset in pezzi più piccoli, chiamati **shard**, che vengono distribuiti su più macchine o cluster.
- In questo modo:
 - ogni macchina lavora solo su una parte dei dati (*scalabilità*),
 - il carico viene distribuito (*load balancing*),
 - se una macchina si rompe, le altre continuano a lavorare (*fault tolerance*).
- Tipi di sharding:
 - **Range-Based**: i dati vengono divisi in base a **intervalli** di valori (es. ID o lettere)
 - **Hash-Based**: i dati vengono distribuiti usando una **funzione hash** su una colonna chiave (es. ID)
 - **Geographic**: i dati vengono divisi in base alla **posizione geografica** (es. server per Europa, server per Asia, server per America)

Vector Storage - Partitioning



- Consiste nel dividere i dati all'interno di una **singola macchina** o database in più sottogruppi logici. Tutti i dati rimangono nello stesso sistema fisico, ma vengono organizzati in modo da migliorare efficienza e prestazioni.
- In questo modo:
 - le query possono interrogare solo i sottogruppi rilevanti,
 - più partizioni possono essere elaborate contemporaneamente (*parallel processing*),
 - si evitano scansioni inutili di dati non pertinenti ottimizzando l'uso delle risorse.
- Tipi di partitioning:
 - **Range-Based**: i dati vengono suddivisi per intervalli di valori
 - **Hash-Based**: come lo sharding, ma all'interno della stessa macchina
 - **List-Based**: i dati vengono suddivisi per insiemi specifici di valori
 - **K-Means**: si effettua un raggruppamento dei dati basato su similarità

Vector Storage - Caching



- Memorizza i dati più usati o recentemente richiesti in una **memoria veloce** (RAM) con l'obiettivo di **ridurre i tempi di accesso** e velocizzare la ricerca dei vettori.
- In questo modo:
 - si può evitare di interrogare tutto il VDB,
 - la risposta è più immediata perché la cache permette di rispondere subito a richieste frequenti o recenti.
- Il **workflow** è il seguente: (1) L'applicazione richiede dei vettori (2) Il sistema controlla la cache: **Hit** se i dati sono in cache (restituiti subito), **Miss** se i dati non sono in cache (lettura dal DB principale) (3) I vettori recuperati vengono memorizzati in cache per richieste future (4) Se la cache è piena, si applica una policy di rimozione (FIFO, LRU, MRU, LFU).
- Di solito, i vettori vengono divisi in partizioni logiche (es. categoria, posizione geografica) e la cache mantiene i dati più rilevanti per ciascuna partizione.

Vector Storage - Replication



- Crea **copie multiple dei dati vettoriali** e le memorizza su nodi o cluster diversi.
- In questo modo:
 - i dati sono sempre accessibili anche se un nodo fallisce,
 - le informazioni non si perdono,
 - le query possono essere servite da più nodi.
- Tipi di replication:
 - **Leader-Follower**: un nodo è un leader e riceve tutte le scritture, gli altri sono follower e ricevono solo gli aggiornamenti dal leader
 - **Multi-Leader**: più nodi sono leader e ognuno riceve scritture in parallelo, gli aggiornamenti sono propagati asincronamente agli altri nodi
 - **Leaderless**: non ci sono leader o follower e qualsiasi nodo può leggere e scrivere

- Si basa sugli algoritmi di Nearest Neighbor Search (NNS) e ha l'obiettivo di trovare il vettore più vicino o più simile a un vettore dato.
- La “vicinanza” è misurata di solito tramite una funzione di dissimilarità (valori più piccoli = più simili).
- Tipi di ricerca:
 - **Exact Nearest Neighbor Search (NNS)**: trova il vettore più vicino in modo preciso
 - **Approximate Nearest Neighbor Search (ANNS)**: permette una piccola approssimazione dei risultati

Esempi di utilizzo



Sistemi di raccomandazione

Le preferenze dell'utente e le caratteristiche dell'oggetto sono rappresentate come vettori ed indicizzate nei VDBs.

Ricerca di immagini e video

I VDBs consentono la ricerca di immagini o video visivamente simili.

Natural Language Processing

I VDBs facilitano la ricerca semantica, la modellazione degli argomenti e il clustering dei documenti.

Fraud Detection

I VDBs possono aiutare nell'identificare pattern e anomalie nelle transazioni finanziarie, agevolando il rilevamento delle frodi.

VDBs esistenti



Vector Databases



Vector Libraries



Vector-Capable NoSQL Databases



Vector-Capable SQL Databases



Text Search Databases



VDBs più popolari



Nome	Caratteristiche
	• Open-source (Zilliz). • Progettato specificamente per ANN search su larga scala. • Supporto a GPU acceleration per migliorare le performance. • Integrabile con ecosistemi Big Data e AI.
	• Cloud-native SaaS (non open-source). • Gestione automatica di scalabilità, replica e aggiornamenti. • Ottimo per applicazioni LLM e AI search. • Query low-latency senza bisogno di configurazioni complesse.
	• Open-source con API REST/GraphQL. • Supporta Hybrid Search: keyword + semantic + vector search insieme. • Estendibile con plugin (es. HuggingFace, OpenAI). • Ottimo per knowledge graph + semantic search.
	• Open-source, scritto in Rust → altissime prestazioni. • Ottimizzato per real-time vector search. • Supporto a payload/metadata filtering integrato. • Focus su semplicità di deploy e API moderne.
	• Open-source, lightweight e super usato come “default” con LLM. • Facile integrazione con LangChain e framework LLM. • Non ottimizzato per dataset enormi ma più adatto a ricerca semantica rapida e progetti piccoli/medi. • Semplice da usare localmente, ottimo per imparare.

FAISS (*Facebook AI Similarity Search*) è una libreria open-source sviluppata da Meta AI e progettata per cercare in modo efficiente in grandi collezioni di vettori ad alta dimensione. Non è un database, ma un motore di indicizzazione e ricerca.

- **Caratteristiche principali**

- Supporta sia Exact Nearest Neighbor (NNS) che Approximate Nearest Neighbor (ANNS).
- Implementa strutture dati avanzate (HNSW, IVF, PQ, ecc.) ottimizzate per velocità e memoria.
- Supporto a GPU - prestazioni molto elevate su dataset enormi.
- Scalabile per miliardi di vettori.

- **Utilizzo pratico**

- Molti Vector DB (es. Milvus, Pinecone, Weaviate) integrano FAISS come motore di ricerca interno.
- Può essere usato direttamente in Python per compiti come ricerca di immagini simili, clustering di testi, recommendation systems.





- **Memorizzazione degli embeddings**
 - Ogni documento può avere un campo embedding.
 - MongoDB salva insieme: dati testuali/strutturati + embedding.
- **Creazione dell'indice vettoriale**
 - Si definisce un indice su quel campo (knnVector).
 - Parametri: dimensione (es. 768), funzione di similarità (cosine, dot product, euclidean).
- **Query vettoriale (Vector Search con Atlas Vector Search)**
 - Inserisco un embedding di query (es. embedding di una frase o immagine).
 - MongoDB trova i documenti con embedding più simili (nearest neighbors).
 - Possibilità di combinare filtri testuali / metadata + ricerca semantica.



- **Memorizzazione degli embeddings**

- Ogni nodo del grafo (es. Product, User, Document) può avere una proprietà embedding.
- Neo4j supporta embedding e vector similarity search tramite il plugin **Neo4j Graph Data Science (GDS)**.

- **Creazione dell'indice vettoriale**

- Si crea un VECTOR INDEX su una proprietà dei nodi.
- Parametri: dimensione, funzione di similarità, configurazioni HNSW per performance.

- **Query vettoriale**

- Con `db.index.vector.queryNodes` passo un embedding di input.
- Neo4j restituisce i nodi più simili (con score di similarità).

- **Integrazione con il grafo**

- Dopo la ricerca vettoriale, posso usare le relazioni del grafo, ad esempio “trova utenti simili a questo embedding e vedi a quali prodotti sono collegati”.



- **Memorizzazione degli embeddings**

- Ogni documento in Elasticsearch può avere un campo di tipo `dense_vector`.
- Questo campo contiene l'embedding (array di float) generato da un modello ML.

- **Creazione dell'indice vettoriale**

- Elasticsearch permette di definire un indice con campo `dense_vector`.
- Supporta diverse metriche di similarità: cosine, dot product, euclidean.
- Algoritmo principale: **HNSW (Hierarchical Navigable Small World)** per ANN.

- **Query vettoriale**

- Si usa il parametro `knn` (k-nearest neighbors) nella query.
- Elasticsearch trova i documenti con embedding più vicini a quello fornito.
- Possibile combinare ricerca full-text + query vettoriale (ottimo per applicazioni ibride come RAG, motori di ricerca semantici).

- **Ruolo nei Vector DB**

- Pre-elaborazione: pulizia e trasformazione dei dati (testo, immagini, audio).
- Generazione embedding: con MLlib o librerie Python/Deep Learning.
- Distribuzione: gestione di miliardi di vettori su cluster.
- Integrazione: invio embeddings a Vector DB (es. Milvus, Pinecone, Weaviate) o indicizzazione con FAISS.

- **Workflow tipico**

1. Dataset enorme (documenti, immagini, log).
2. Spark calcola embeddings in parallelo.
3. Embeddings sono salvati in un Vector DB.
4. Vector DB usato per retrieval.

- **Vantaggi**

- Scalabilità massiva.
- Integrazione con ecosistema Big Data (HDFS, Kafka, Delta Lake).
- Pipeline personalizzabili per AI + ricerca semantica.

Retrieval-Augmented Generation (RAG) è un'architettura che combina LLM + Vector DB per migliorare le risposte con conoscenza esterna.

- **Embedding della query**

- L'utente fa una domanda ad un LLM che viene convertita in un vettore (embedding) usando lo stesso modello di embedding usato per indicizzare i documenti presenti nel VDB.

- **Query al VDB**

- Il vettore della query viene inviato al VDB che esegue una similarity search (ANN/NNS) tra la query e i vettori che aveva già indicizzati e restituisce i documenti più simili (top-k).

- **Recupero del contesto**

- I documenti trovati vengono “riconvertiti” in testo leggibile.
- Questi testi diventano il contesto da aggiungere al prompt dell'LLM.

- **Prompt arricchito**

- L'LLM riceve la domanda originale dell'utente + i documenti recuperati dal Vector DB.
- L'LLM genera una risposta che combina conoscenza propria + informazioni dal VDB.